

Principal Component Analysis Algorithm (PCA)

<https://byjus.com/maths/correlation/>

<https://towardsdatascience.com/a-step-by-step-implementation-of-principal-component-analysis-5520cc6cd598>

https://github.com/AdityaDutt/PCATutorial/blob/main/PCA_tutorial.ipynb

<https://github.com/patrickloeber/MLfromscratch/blob/master/mlfromscratch/pca.py>

Introduction

Principal Component Analysis or PCA is a commonly used dimensionality reduction method. It works by computing the principal components and performing a change of basis. It retains the data in the direction of maximum variance. The reduced features are uncorrelated with each other. These features can be used for unsupervised clustering and classification. To reduce dimensionality, autoencoder is another commonly used method. But, the latent space of the autoencoder is not necessarily uncorrelated. While PCA guarantees that all features are uncorrelated with each other.

Firstly, PCA computes the covariance matrix. Then we find the eigen vectors and eigen values of the covariance matrix. After that, we project the data along the eigen vectors. If the original data has a dimensionality of n , we can reduce dimensions to k , such that $k \leq n$.

In this tutorial, we will implement PCA from scratch and understand the significance of each step.

Implementation

Firstly, import libraries.

```
In [1]: use strict;
use warnings;
use Data::Dump qw(dump);
use AI::MXNet qw(nd);
use sml qw(show_plot);
IPerl->load_plugin('Chart::Plotly');
```

Step 1: Fetch the data from a dataset

We obtain data from Iris Dataset, which has 4 columns. We use the function previously defined in the book.

```
In [3]: my ($dataset, $header) = sml->load_csv('../data/iris.csv');
my ($lookup, $rlookup) = sml->str_column_to_int($dataset, -1);
$dataset = nd->array($dataset);
```

```
Out[3]: <AI::MXNet::NDArray 150x5 @cpu(0)>
```

```
In [4]: printf "%d-%s ", $_, $header->[$_] for (0 .. $#header);
printf "%s\n", dump $lookup, $rlookup;
```

```
0-SepallengthCm 1-SepalWidthCm 2-PetalLengthCm 3-PetalWidthCm 4-Species (
{ "Iris-setosa" => 0, "Iris-versicolor" => 1, "Iris-virginica" => 2 },
{ "0" => "Iris-setosa", "1" => "Iris-versicolor", "2" => "Iris-virginica" },
)
```



```

Matriz Min-Max por columna:[[4.3000 7.9000]
[2.0000 4.4000]
[1.0000 6.9000]
[0.1000 2.5000]]
<AI::MXNet::NDArray 4x2 @cpu(0)> AI::MXNet::NDArray float32
Normalized:[[0.2222 0.6250 0.0678 0.0417      0]
[0.1667 0.4167 0.0678 0.0417      0]
[0.1111 0.5000 0.0508 0.0417      0]
[0.0833 0.4583 0.0847 0.0417      0]
[0.1944 0.6667 0.0678 0.0417      0]]
<AI::MXNet::NDArray 5x5 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[7]: 1

Step 3: Mean Centering

Mean centering is done to ensure that the first Principal Component is in the direction of maximum variance.

We will do mean centering by subtracting mean from all features or channels.

```

In [8]: my $mean_data = $X_normalized - nd->mean($X_normalized, axis=>0);
        print $mean_data->slice([0, 5])>asstr;

```

```

[[-0.2065  0.1858 -0.3998 -0.4161]
 [-0.2620 -0.0225 -0.3998 -0.4161]
 [-0.3176  0.0608 -0.4167 -0.4161]
 [-0.3454  0.0192 -0.3828 -0.4161]
 [-0.2343  0.2275 -0.3998 -0.4161]]

```

Out[8]: 1

Step 4: Compute the covariance matrix

Now, we compute the covariance of all features dimensions. Every covariance matrix is symmetric and positive semi-definite. It has orthogonal eigen vectors.

```

In [9]: # Compute covariance matrix
        my $cov_matrix = nd->cov($mean_data->T, dtype => 'float32');
        # $cov_matrix = nd->_npi_around($cov_matrix, 3);
        nd->print("Covariance matrix:", $cov_matrix);

```

```

Covariance matrix:[[ 0.0529 -0.0045  0.0600  0.0598]
 [-0.0045  0.0326 -0.0227 -0.0205]
 [ 0.0600 -0.0227  0.0894  0.0916]
 [ 0.0598 -0.0205  0.0916  0.1011]]
<AI::MXNet::NDArray 4x4 @cpu(0)> AI::MXNet::NDArray float32

```

Out[9]: 1

Step 5: Compute eigen vectors of the covariance matrix

Now, we perform the eigen decomposition of the covariance matrix and we will get eigen vectors and eigen values. (The number of eigen values/ vectors will be the same as the number of features/ channels.)

Eigenvalues y Eigenvectors

Consideramos la matriz de dimensiones 2×2 :

$$A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$$

1. Cálculo de los Valores Propios (Eigenvalues)

Para encontrar los valores propios λ , resolvemos la ecuación característica $\det(A - \lambda I) = 0$:

$$\det \begin{pmatrix} 0.7 - \lambda & 0.3 \\ 0.4 & 0.6 - \lambda \end{pmatrix} = 0$$

Desarrollando el determinante:

$$(0.7 - \lambda)(0.6 - \lambda) - (0.3 \times 0.4) = 0$$

$$\lambda^2 - 1.3\lambda + 0.42 - 0.12 = 0$$

$$\lambda^2 - 1.3\lambda + 0.30 = 0$$

Factorizando el polinomio cuadrático:

$$(\lambda - 1)(\lambda - 0.3) = 0$$

Por lo tanto, los valores propios son:

$$\lambda_1 = 1, \quad \lambda_2 = 0.3$$

2. Cálculo de los Vectores Propios Derechos (Right Eigenvectors)

Resolvemos el sistema $(A - \lambda I)v = 0$ para cada valor propio, donde $v = \begin{pmatrix} x \\ y \end{pmatrix}$.

Caso $\lambda_1 = 1$:

$$(A - 1I)v = \begin{pmatrix} -0.3 & 0.3 \\ 0.4 & -0.4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Ambas filas producen la relación $-0.3x + 0.3y = 0 \implies x = y$.

Elegimos el vector base $v_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$. Para normalizarlo bajo la norma Euclídea (

$$\|v_1\| = \sqrt{1^2 + 1^2} = \sqrt{2}):$$

$$v_1 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \approx \begin{pmatrix} 0.7071 \\ 0.7071 \end{pmatrix}$$

Caso $\lambda_2 = 0.3$:

$$(A - 0.3I)v = \begin{pmatrix} 0.4 & 0.3 \\ 0.4 & 0.3 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

La ecuación resultante es $0.4x + 0.3y = 0 \implies x = -0.75y$.

Si fijamos $y = 0.8$, obtenemos $x = -0.6$. El vector derecho correspondiente es:

$$v_2 = \begin{pmatrix} -0.6000 \\ 0.8000 \end{pmatrix}$$

Matriz de Resultados

$$\text{vals} = [1 \quad 0.3000], \quad \text{vector} = \begin{bmatrix} 0.7071 & -0.6000 \\ 0.7071 & 0.8000 \end{bmatrix}$$

```
In [10]: # Perform eigen decomposition of covariance matrix
# my ($eig_vals, $eig_vec) = nd->linalg->eig($cov_matrix);
my ($eig_vals, $eig_vec) = @{nd->npi_eig($cov_matrix)};
nd->printf("Eigen values:\n%.6f\n", $eig_vals);
nd->printf("Eigen vectors:\n%.6f\n", $eig_vec);

# Eigen values [0.23231168 0.03239279 0.00963728 0.00175337]

# Eigen vectors
# [[ 0.4252893 -0.42100611 -0.71434339 0.36276216]
# [-0.14612219 -0.90470509 0.33510219 -0.21877734]
# [ 0.61610511 0.06432645 -0.06825927 -0.78205964]
# [ 0.64667752 0.01116485 0.61054133 0.45708076]]
```

Eigen values:

[0.232312 0.032393 0.009637 0.001753]

<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32

Eigen vectors:

[[-0.425289 0.421006 0.714343 0.362763]

[0.146122 0.904705 -0.335102 -0.218778]

[-0.616105 -0.064327 0.068260 -0.782060]

[-0.646677 -0.011164 -0.610542 0.457080]]

<AI::MXNet::NDArray 4x4 @cpu(0)> AI::MXNet::NDArray float32

Out[10]: 1

Each eigen vector represents a direction of variance. The eigen vector corresponding to the largest eigen value will give the direction of maximum variance. This is the first principal component. Then, the eigen vector corresponding to the 2nd largest eigen value will give the direction of the second largest variance. This is the second principal component. And, so on.

So, we need to sort the eigen vectors according to descending order of eigen values. Remember, all eigen vectors are orthogonal to each other.

```
In [11]: # Sort eigen values and vectors in descending order
my $sort_idx = nd->argsort($eig_vals)->reverse(axis=>0);
my $sort_eig_vals = nd->take($eig_vals, $sort_idx);
my $sort_eig_vec = nd->take($eig_vec->T, $sort_idx);

nd->printf("Sorted Eigen values:\n%.6f\n", $sort_eig_vals);
nd->printf("Sorted Eigen vectors:\n%.6f\n", $sort_eig_vec);

# Sorted Eigen values [0.23231168 0.03239279 0.00963728 0.00175337]

# Sorted Eigen vectors
# [[ 0.4252893 -0.14612219 0.61610511 0.64667752]
# [-0.42100611 -0.90470509 0.06432645 0.01116485]
# [-0.71434339 0.33510219 -0.06825927 0.61054133]
# [ 0.36276216 -0.21877734 -0.78205964 0.45708076]]
```

Sorted Eigen values:

[0.232312 0.032393 0.009637 0.001753]

<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32

Sorted Eigen vectors:

[[-0.425289 0.146122 -0.616105 -0.646677]

[0.421006 0.904705 -0.064327 -0.011164]

[0.714343 -0.335102 0.068260 -0.610542]

[0.362763 -0.218778 -0.782060 0.457080]]

<AI::MXNet::NDArray 4x4 @cpu(0)> AI::MXNet::NDArray float32

Out[11]: 1

Step 6: Compute the explained variance and select N components

We can select top k eigen vectors based on how much compression do we want.

The optimal way of selecting the number of components is to compute the explained variance of each feature. We compute explained variance by dividing the eigen values by the sum of all eigen values. Then, we take the cumulative sum of all eigen values.

```
In [12]: # Get explained variance
my $explained_variance = $sort_eig_vals / $sort_eig_vals->sum();
nd->print("Explained variance:", $explained_variance);
my $cumulative_variance = nd->cumsum($explained_variance);
nd->print("cumulative_variance:", $cumulative_variance);

# explained_variance: [0.84141901 0.11732474 0.03490564 0.00635061]
# cumulative_variance: [0.84141901 0.95874375 0.99364939 1.          ]
```

```
Explained variance:[0.8414 0.1173 0.0349 0.0064]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32
cumulative_variance:[0.8414 0.9587 0.9936 1.0000]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32
```

Out[12]: 1

Step 7: Transform Data using eigen vectors

Now, we will take the dot product of our data with the eigen vectors to get projections of our data in the direction of these eigen vectors.

```
In [13]: my $pca_data = nd->dot($mean_data, $sort_eig_vec);
nd->printf("Transformed data:\n%.6f\n", $pca_data->slice([0, 5]));

# Transformed data:
# [[-0.03142478 -0.1808823 0.23745169 -0.56572707]
# [ 0.03265764 0.01571582 0.18982228 -0.60397961]
# [-0.01394587 -0.05723807 0.1621117 -0.64932389]
# [-0.03243257 -0.00412367 0.14000352 -0.64705604]
# [-0.06078029 -0.2145194 0.22301793 -0.58322513]]
```

```
Transformed data:
[[-0.270474 0.362954 0.413395 0.185335]
 [-0.334556 0.166356 0.461025 0.223587]
 [-0.287953 0.239310 0.488735 0.268932]
 [-0.269466 0.186195 0.510843 0.266664]
 [-0.241119 0.396591 0.427829 0.202833]]
<AI::MXNet::NDArray 5x4 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[13]: 1

```
In [14]: package PCA{
  use strict;
  use warnings;
  use AI::MXNet qw(nd);

  sub new{
    my ($class, %args) = (shift, n_components => undef, @_);

    my $self = {
```

```

        n_components => $args{n_components},
        components   => undef,
        mean         => undef,
        variance_share => undef,
        chosen_columns => undef,
    };

    return bless($self, $class);
}

sub fit{
    my ($self, $X) = @_;

    # 1. Copia no destructiva y centrado de datos (Evita mutar el X original)
    $self->{mean} = nd->mean($X, axis => 0);
    my $X_centered = $X - $self->{mean};

    # Si n_components es undef, asignamos dinámicamente el máximo posible
    $self->{n_components} //= $X_centered->shape->[1];

    # 2. Matriz de Covarianza balanceada por (N - 1)
    my $cov_matrix = nd->cov($X_centered->T, dtype => 'float32');

    # Eigenvalues, eigenvectors
    # my ($eig_vals, $eig_vec) = nd->linalg->eig($cov_matrix);
    my ($eig_vals, $eig_vec) = @{nd->npi_eig($cov_matrix)};

    # Sort eigen values and vectors in descending order
    my $sort_idx = nd->argsort($eig_vals)->reverse(axis=>0);
    my $sort_eig_vals = nd->take($eig_vals, $sort_idx);
    my $sort_eig_vec = nd->take($eig_vec->T, $sort_idx);

    # store principal components & variance
    $self->{chosen_columns} = $sort_idx->slice([0, $self->{n_components}]);
    $self->{components} = $sort_eig_vec->slice([0, $self->{n_components}]);
    $self->{variance_share} = nd->sum($sort_eig_vals->slice([0, $self->{n_components}])
        / nd->sum($sort_eig_vals);
}

sub transform {
    my ($self, $X) = @_;
    die "The PCA model has not yet been trained. First run 'fit()'."
        unless defined $self->{components};

    # Operación no destructiva de centrado para la proyección
    my $X_centered = $X - $self->{mean};

    return nd->dot($X_centered, $self->{components}->T);
}

# =====
# Model parameters persistency
# =====

sub save_model {
    my ($self, $filename) = @_;
    die "No hay parámetros entrenados para guardar." if !defined $self->{components};
    $filename //= 'pca_model.ndarray';

    # Creamos un hash intermedio de tensores compatibles con nd->save
    my $save_data = {
        'n_components' => nd->array([$self->{n_components}]),
        'components'   => $self->{components}->sever, # materializes slice
        'mean'         => $self->{mean},
        'variance_share' => $self->{variance_share},
        'chosen_columns' => $self->{chosen_columns}->sever, # materializes slice
    };

```

```

};

# Guarda el diccionario estructurado en un archivo binario
nd->save($filename, $save_data);

return 1;
}

sub load_model {
    my ($self, $filename) = @_;
    $filename //= 'pca_model.ndarray';
    die "El archivo de modelo '$filename' no existe." unless -e $filename;

    # Retorna una referencia a un hash con los mismos nombres de llaves
    my $loaded_data = nd->load($filename);

    # Alimenta las propiedades del objeto con los NDArrays recuperados
    $self->{n_components} = int($loaded_data->{'n_components'}->asscalar);
    $self->{components} = $loaded_data->{'components'};
    $self->{mean} = $loaded_data->{'mean'};
    $self->{variance_share} = $loaded_data->{'variance_share'};
    $self->{chosen_columns} = $loaded_data->{'chosen_columns'};

    return 1;
}

1;
};

```

Out[14]: 1

In [15]: `nd->printf("Normalized data:\n%.6f\n", $X_normalized->slice(end=>5));`

```

Normalized data:
[[0.222222 0.625000 0.067797 0.041667]
 [0.166667 0.416667 0.067797 0.041667]
 [0.111111 0.500000 0.050847 0.041667]
 [0.083333 0.458333 0.084746 0.041667]
 [0.194444 0.666667 0.067797 0.041667]]
<AI::MXNet::NDArray 5x4 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[15]: 1

In [16]: `# Project the dataset onto the 2 primary principal components`
`my $pca = new PCA(n_components=>2);`
`$pca->fit($X_normalized);`
`my $X_PCA = $pca->transform($X_normalized);`

Out[16]: <AI::MXNet::NDArray 150x2 @cpu(0)>

In [17]: `printf "Shape of X: %s\n", dump $X_normalized->shape;`
`printf "Shape of transformed X: %s\n", dump $X_PCA->shape;`
`printf "Variance share: %s\n", $pca->{variance_share}->asstr('%.2f');`
`printf "Chosen columns: %s\n", $pca->{chosen_columns}->asstr;`

```

# Shape of X: (150, 4)
# Shape of transformed X: (150, 2)
# Variance share: 0.9587437487644828
# Chosen columns: [0 1]

```

```

Shape of X: [150, 4]
Shape of transformed X: [150, 2]
Variance share: [0.96]
Chosen columns: [0 1]

```

Out[17]: 1


```
In [18]: nd->printf("PCA data:\n%.6f\n", $X_PCA->slice(end=>5));
# [[-0.63036125 -0.11155626]
# [-0.62354631 0.1003132 ]
# [-0.6697928 0.04722006]
# [-0.65463305 0.09879127]
# [-0.64826327 -0.13755769]]
```

PCA data:

```
[[ 0.630361 0.111556]
 [ 0.623546 -0.100313]
 [ 0.669793 -0.047220]
 [ 0.654633 -0.098791]
 [ 0.648263 0.137558]]
```

<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[18]: 1

```
In [19]: my $X1_PCA = $X_PCA->slice(':', 0);
my $X2_PCA = $X_PCA->slice(':', 1);
nd->print("Pearson Correlation X1_PCA x X2_PCA:", nd->corrcoef($X1_PCA, $X2_PCA));
# Pearson Correlation X1_PCA x X2_PCA:
# [[ 1 0.0000]
# [0.0000 1]]
# <AI::MXNet::NDArray 2x2 @cpu(0)> float64
```

Pearson Correlation X1_PCA x X2_PCA: [[1.0000 0.0000]
[0.0000 1.0000]]

<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64

Out[19]: 1

```
In [20]: # Plotting the result of the PCA data transformation

my $color_scale = [
    [0, 'green'],
    [0.5, 'purple'],
    [1, 'orange']
];

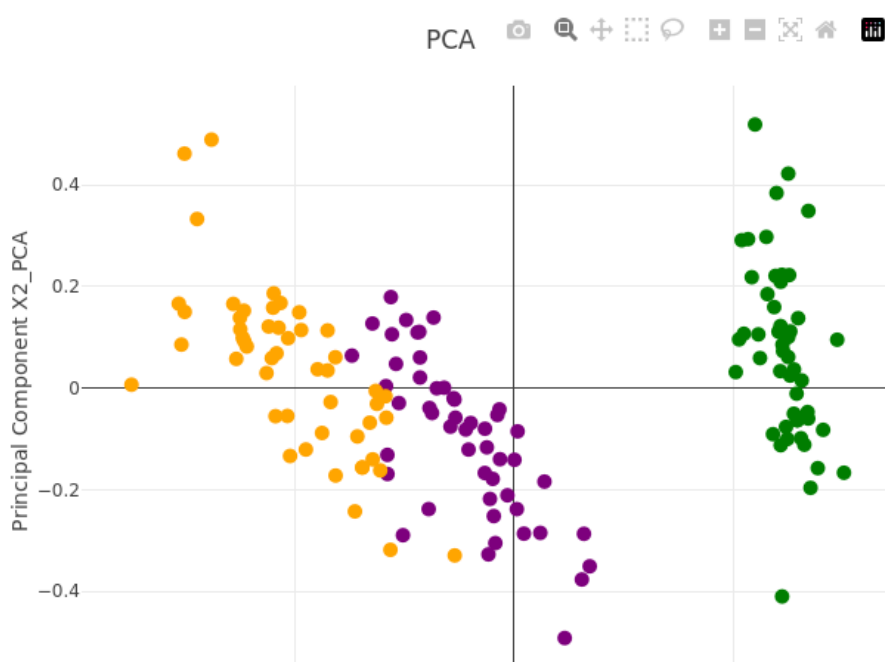
my $trace = new Chart::Plotly::Trace::Scatter(
    x => $X1_PCA->asperl,
    y => $X2_PCA->asperl,
    mode => 'markers',
    marker => {
        color => $y->asperl,
        colorscale => $color_scale,
        cmin => $y->min,
        cmax => $y->max,
        size => 10
    }
);

my $layout = {title => { text => 'PCA' },
    xaxis => { title => 'Principal Component X1_PCA' },
    yaxis => { title => 'Principal Component X2_PCA' },
    width => 600, height => 500,
    margin => { l => 50, r => 0, t => 50, b => 50 } };

my $plot = new Chart::Plotly::Plot(traces => [$trace],
    layout => $layout);

IPerl->display($plot);
```

```
In [22]: sml->embed_plot($plot, width=>900, height=>450);
```



How overlapping were those columns before PCA?

In [23]: *# Let's first investigate the changes in the columns chosen by the PCA algorithm.*

```
my @chosen_columns = $pca->{chosen_columns}->tolist;
printf "Chosen columns: %s", dump @chosen_columns;

my ($X0_normalized, $X1_normalized) = @{$X_normalized->T}[@chosen_columns];
```

Chosen columns: (0, 1)

Out[23]: <AI::MXNet::NDArray 150 @cpu(0)><AI::MXNet::NDArray 150 @cpu(0)>

In [24]: *# Plotting the normalized data before PCA projection*

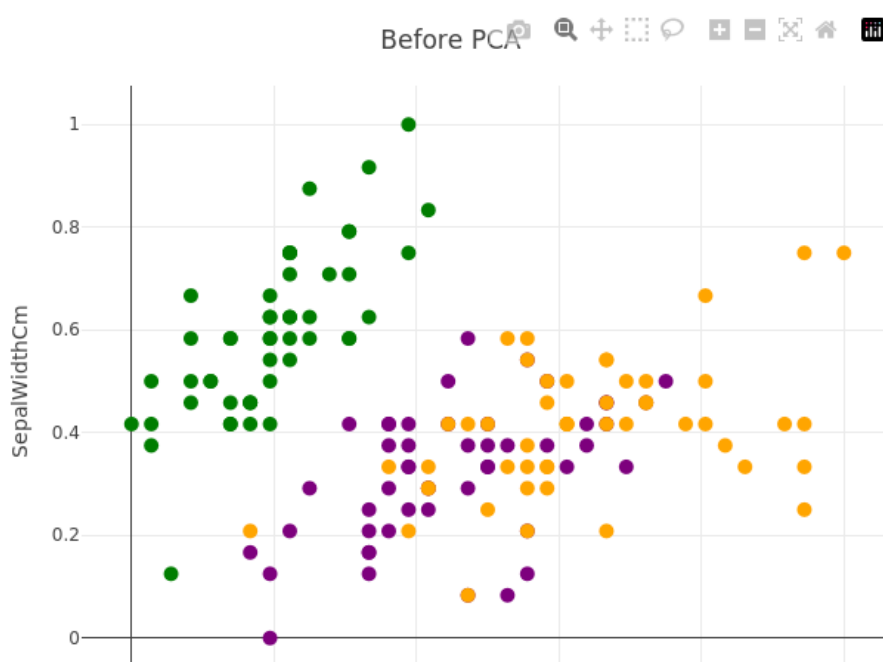
```
my $trace = new Chart::Plotly::Trace::Scatter(
    x      => $X0_normalized->aspdf,
    y      => $X1_normalized->aspdf,
    mode   => 'markers',
    marker => {
        color      => $y->aspdf,
        colorscale => $color_scale,
        cmin       => $y->min,
        cmax       => $y->max,
        size       => 10
    }
);

my $layout = {title => { text => 'Before PCA' },
    xaxis => { title => $header->[$chosen_columns[0]] },
    yaxis => { title => $header->[$chosen_columns[1]] },
    width  => 600, height => 500,
    margin => { l => 50, r => 0, t => 50, b => 50 } };

my $plot = new Chart::Plotly::Plot(traces => [$trace],
    layout => $layout);

IPerl->display($plot);
```

In [25]: `sml->embed_plot($plot, width=>900, height=>450);`



We basically find an intense overlap between the two features regarding the purple and orange points. The lack of separability is a source of errors in Machine Learning.

Changes in the Pearson Correlation Coefficients

Now let's check how the Pearson Correlation Coefficients have been affected by PCA projection.

```
In [26]: my $PCA = nd->concat($X_PCA, $y->expand_dims(axis=>1));
nd->print("Normalized:", $PCA->slice([0, 5]));
```

```
Normalized:[[ 0.6304  0.1116   0]
 [ 0.6235 -0.1003   0]
 [ 0.6698 -0.0472   0]
 [ 0.6546 -0.0988   0]
 [ 0.6483  0.1376   0]]
<AI::MXNet::NDArray 5x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[26]: 1

```
In [27]: # Coeficientes before PCA
```

```
my $corrcoef_before = nd->corrcoef($normalized->T);
printf "Pearson Correlation coefficients of the Iris dataset:\n%s\n",
    $corrcoef_before->asstr;
```

```
Pearson Correlation coefficients of the Iris dataset:
[[ 1.0000 -0.1094  0.8718  0.8180  0.7826]
 [-0.1094  1.0000 -0.4205 -0.3565 -0.4194]
 [ 0.8718 -0.4205  1.0000  0.9628  0.9490]
 [ 0.8180 -0.3565  0.9628  1.0000  0.9565]
 [ 0.7826 -0.4194  0.9490  0.9565  1.0000]]
```

Out[27]: 1

```
In [28]: # Show a heatmap out of the Pearson Correlation coefficients of X
$trace = new Chart::Plotly::Trace::Heatmap(
    x => $header,
    y => $header,
    z => $corrcoef_before->aspdf,
    colorscale => 'Jet', # Electric Greens Greys Hot Jet Picnic Portland Reds YlOrRd
    # https://plotly.com/javascript/colormaps/#hot-colorscale
);
```

```

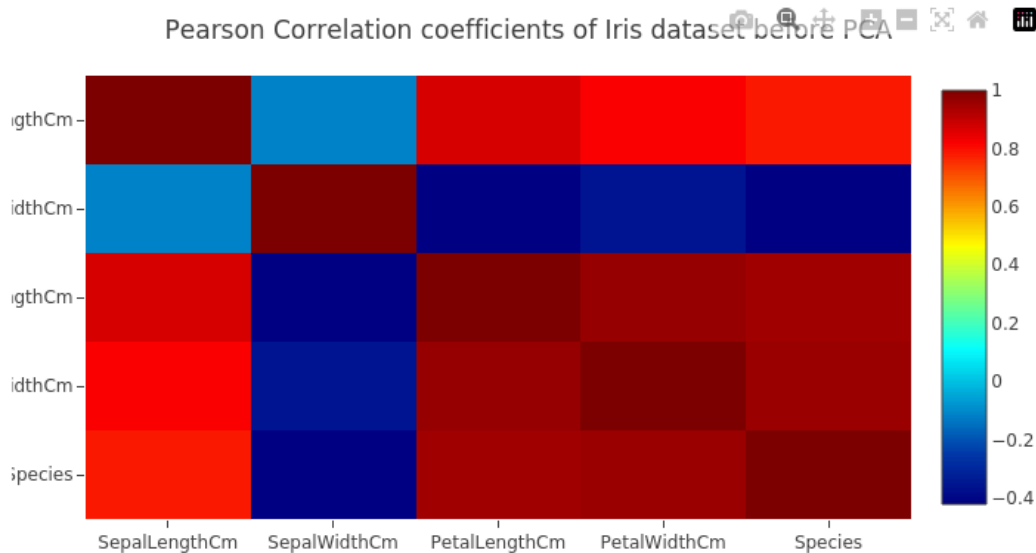
$layout = {title => { text => 'Pearson Correlation coefficients of Iris dataset before PCA',
  yaxis => { autorange => "reversed" }, # Flip so lower frequencies are at top
  width => 700, height => 400,
  margin => { l => 50, r => 0, t => 50, b => 50 }};

$plot = new Chart::Plotly::Plot(traces => [$trace],
  layout => $layout);

IPerl->display($plot);

```

In [29]: `sml->embed_plot($plot, width=>900, height=>450);`



In [30]: `my $corrcoef_after = nd->corrcoef($PCA->T);`
`printf "Pearson Correlation coefficients of the Iris dataset:\n%s\n",`
`$corrcoef_after->asstr;`

Pearson Correlation coefficients of the Iris dataset:

```

[[ 1.0000  0.0000 -0.9527]
 [ 0.0000  1.0000 -0.0802]
 [-0.9527 -0.0802  1.0000]]

```

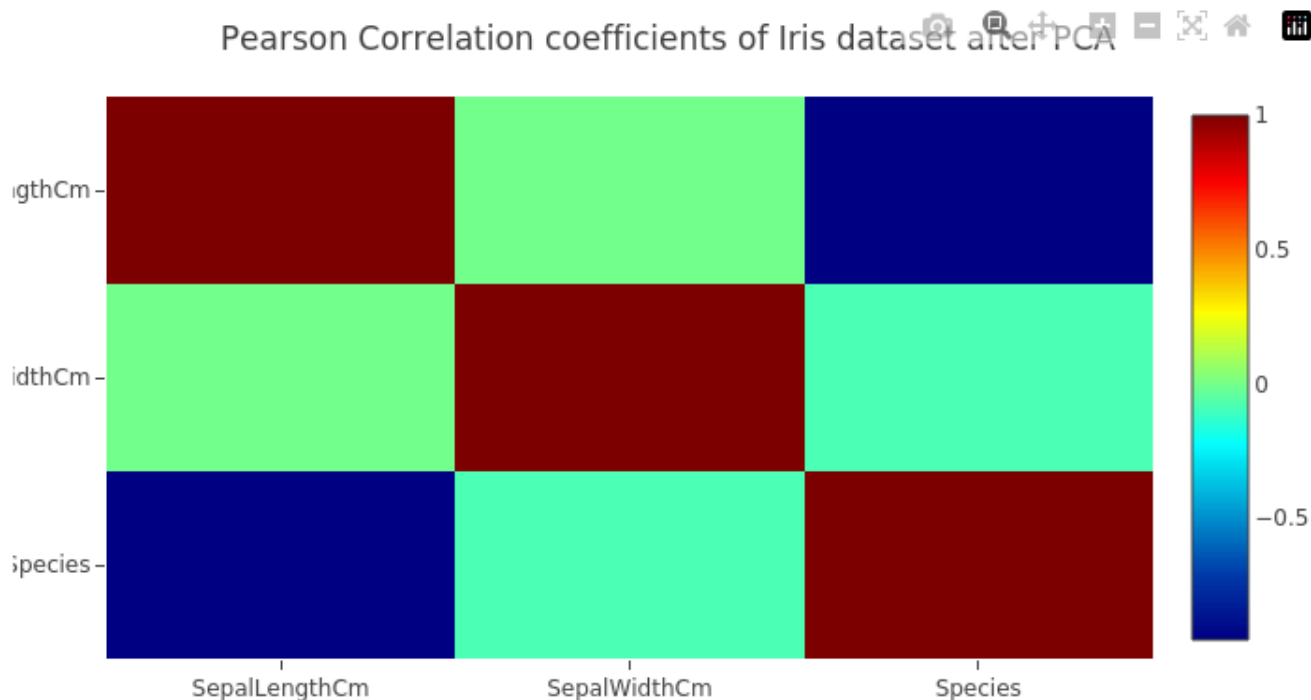
Out[30]: 1

In [31]: `my $squeezed_header = [@header[@chosen_columns, -1]];`

Out[31]: ARRAY(0x55e16234f770)

In [32]: `# Show a heatmap out of the Pearson Correlation coefficients of X`
`$trace = new Chart::Plotly::Trace::Heatmap(`
 `x => $squeezed_header,`
 `y => $squeezed_header,`
 `z => $corrcoef_after->asddl,`
 `colorscale => 'Jet', # Electric Greens Greys Hot Jet Picnic Portland Reds YlOrRd`
 `# https://plotly.com/javascript/colorscale/#hot-colorscale`
`);`
`$layout = {title => { text => 'Pearson Correlation coefficients of Iris dataset after PCA',`
 `yaxis => { autorange => "reversed" }, # Flip so lower frequencies are at top`
 `width => 700, height => 400,`
 `margin => { l => 50, r => 0, t => 50, b => 50 }};`
`$plot = new Chart::Plotly::Plot(traces => [$trace],`
 `layout => $layout);`
`IPerl->display($plot);`

```
In [33]: sml->embed_plot($plot, width=>700, height=>450);
```



Saving and restoring a PCA model

```
In [34]: # Save the object's state to disk
$pca->save_model('modelo_pca_iris.ndarray');
```

Out[34]: 1

```
In [35]: # --- In a clean session or new code block ---
# Instantiate an empty object without hyperparameters
my $pca_restored = new PCA();
$pca_restored->load_model('modelo_pca_iris.ndarray');
```

Out[35]: 1

```
In [36]: # Project over a the data
my $X_PCA = $pca_restored->transform($X_normalized);
nd->print("Projected data:\n", $X_PCA->slice([0, 5]));
```

Projected data:

```
[[ 0.6304  0.1116]
 [ 0.6235 -0.1003]
 [ 0.6698 -0.0472]
 [ 0.6546 -0.0988]
 [ 0.6483  0.1376]]
```

<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[36]: 1

In []: